Préparé par

Samir Ouhmmou et Taoufik Mansouri

Jenkins

Pipeline CI/CD et Jenkinsfile expliqué en détail



Installation ▪ Jenkinsfile ▪ Labs ▪ Commandes ▪ Dépannage

24 octobre 2026

Plan de la présentation

1. Définitions : CI/CD et Jenkins

2. Démarche du lab

3. Installation et configuration

4. Les pipelines

5. Le Jenkinsfile en détail

6. Labs pratiques

7. Fonctions avancées

8. Sécurité et bonnes pratiques

9. Commandes de référence

10. Dépannage

Plan

1. Définitions : CI/CD et Jenkins

2. Démarche du lab

3. Installation et configuration

4. Les pipelines

5. Le Jenkinsfile en détail

6. Labs pratiques

7. Fonctions avancées

8. Sécurité et bonnes pratiques

9. Commandes de référence

10. Dépannage

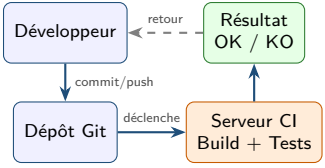
Qu'est-ce que l'intégration continue (CI) ?

Définition

L'**intégration continue** est une pratique où les développeurs fusionnent leur code très fréquemment dans un dépôt commun. À **chaque modification**, le projet est **compilé et testé automatiquement**.

Objectifs

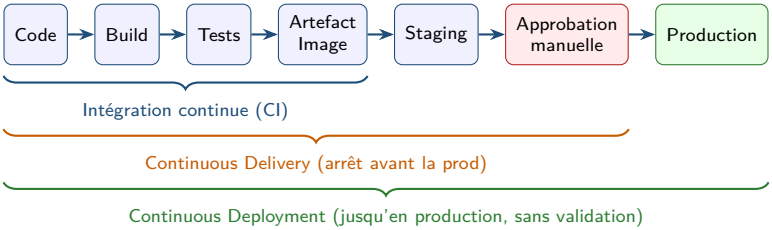
- Détecter les erreurs le plus tôt possible
- Éviter les « conflits d'intégration » tardifs
- Garder une branche principale toujours saine
- Donner un retour rapide aux développeurs



Qu'est-ce que la livraison / le déploiement continu (CD) ?

Continuous Delivery L'application validée est prête à être mise en production (artefact, image Docker). La mise en production reste déclenchée manuellement.

Continuous Deployment La mise en production est entièrement automatique si tous les tests passent.



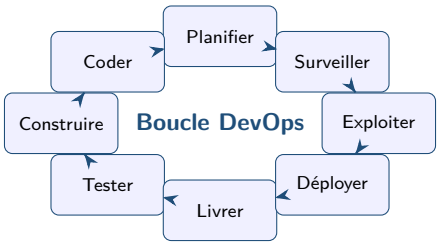
Pourquoi faire du CI/CD ?

Sans CI/CD

- Builds manuels, sources d'erreurs
- Bugs découverts tard
- Déploiements longs et risqués
- « Ça marche sur ma machine »

Avec CI/CD

- Processus automatique et reproductible
- Retour rapide (minutes)
- Livraisons fréquentes et petites
- Qualité mesurée : tests, rapports, traçabilité



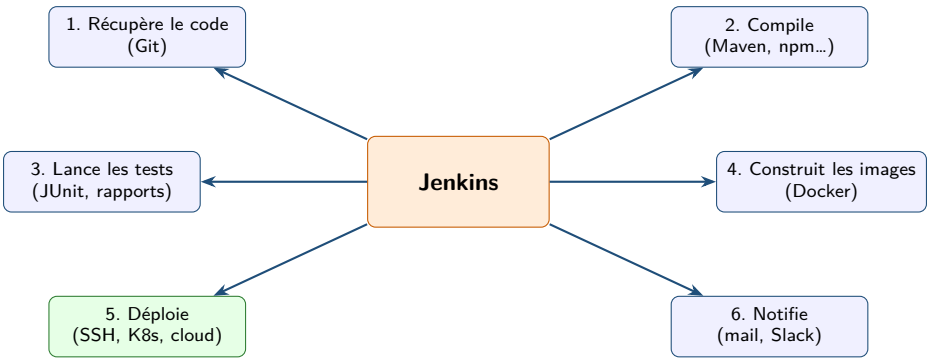
Qu'est-ce que Jenkins ?

Définition

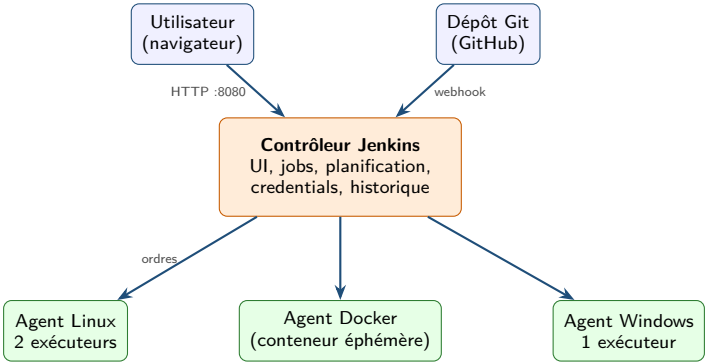
Jenkins est un serveur d'automatisation **open source** écrit en Java. Il exécute automatiquement les tâches du cycle de vie d'un logiciel : compilation, tests, packaging, déploiement, notifications.

- Interface web + configuration *as code* via le Jenkinsfile
- Plus de **1800 plugins** : Git, GitHub, Docker, Kubernetes, SonarQube, Slack, AWS, Ansible...
- Architecture **distribuée** : un contrôleur, plusieurs agents
- Gratuit, très répandu, communauté très active

Que fait Jenkins concrètement ?



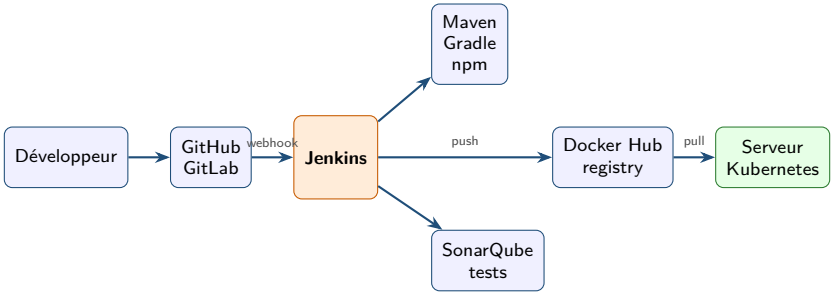
Architecture de Jenkins : contrôleur et agents



Vocabulaire indispensable

Job / Projet	Tâche configurée dans Jenkins (Freestyle, Pipeline, Multibranch…).
Build	Une exécution d'un job, numérotée (#1, #2…).
Pipeline	Suite d'étapes automatisées décrite sous forme de code.
Jenkinsfile	Fichier (syntaxe Groovy) versionné avec le code, qui décrit le pipeline.
Stage / Step	Grande phase du pipeline / action élémentaire (`sh`, `echo`…).
Agent / Node	Machine ou conteneur qui exécute le build.
Executor	Emplacement de build disponible sur un nœud.
Workspace	Dossier de travail sur l'agent (code + fichiers générés).
Artefact	Fichier produit par un build (JAR, ZIP, rapport).
Credential	Secret stocké chiffré (mot de passe, token, clé SSH).
Webhook / Trigger	Appel HTTP de GitHub / mécanisme de déclenchement.

Jenkins dans la chaîne d'outils DevOps



Plan

1. Définitions : CI/CD et Jenkins

2. Démarche du lab

3. Installation et configuration

4. Les pipelines

5. Le Jenkinsfile en détail

6. Labs pratiques

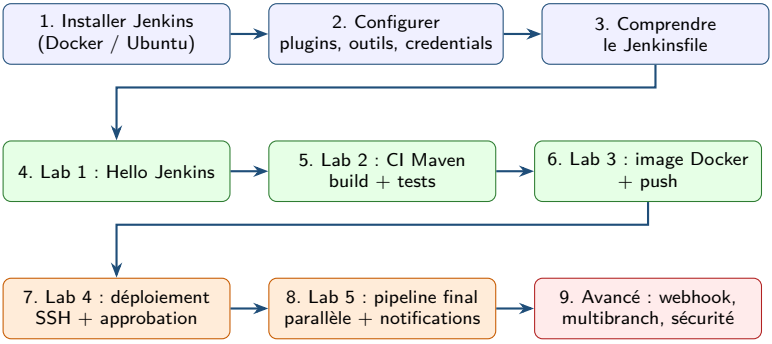
7. Fonctions avancées

8. Sécurité et bonnes pratiques

9. Commandes de référence

10. Dépannage

Feuille de route : notre démarche



Prérequis et projet fil rouge

Prérequis

- Linux (Ubuntu 22.04/24.04) ou Docker Desktop
- 2 Go de RAM libres, 10 Go de disque
- Docker installé : `docker --version`
- Git + compte GitHub
- Notions de terminal Linux

Projet : `demo-app` (Java/Maven)

```
demo-app/
|-- Jenkinsfile
|-- Dockerfile
|-- pom.xml
`-- src/
    |-- main/java/com/example/App.java
    `-- test/java/com/example/AppTest.java
```

Création du projet Maven

```
mvn archetype:generate -DgroupId=com.example -DartifactId=demo-app \
    -DarchetypeArtifactId=maven-archetype-quickstart \
    -DarchetypeVersion=1.4 -DinteractiveMode=false
cd demo-app
git init
git add .
git commit -m "Initial commit"
```

- `archetype:generate` : génère un squelette de projet à partir d'un modèle.
- `-DgroupId`, `-DartifactId` : identifiants Maven (organisation, nom du module).
- `-DinteractiveMode=false` : n'attend aucune saisie.
- `git init` : crée le dépôt local ; `git add .` : met tout en index ; `git commit -m` : premier commit.

Plan

1. Définitions : CI/CD et Jenkins

2. Démarche du lab

3. Installation et configuration

4. Les pipelines

5. Le Jenkinsfile en détail

6. Labs pratiques

7. Fonctions avancées

8. Sécurité et bonnes pratiques

9. Commandes de référence

10. Dépannage

Installer Jenkins avec Docker

```
docker run -d --name jenkins \
  -p 8080:8080 -p 50000:50000 \
  -v jenkins_home:/var/jenkins_home \
  -v /var/run/docker.sock:/var/run/docker.sock \
  --restart unless-stopped \
  jenkins/jenkins:lts-jdk17
```

`-d`	Conteneur en arrière-plan (*detached*).
`--name jenkins`	Nom du conteneur.
`-p 8080:8080`	Port de l'interface web.
`-p 50000:50000`	Port de connexion des agents distants.
`-v jenkins_home:...`	Volume persistant : jobs et plugins conservés.
`-v ...docker.sock`	Donne accès au Docker de l'hôte (lab uniquement).
`--restart unless-stopped`	Redémarrage automatique après un reboot.
`jenkins/jenkins:lts-jdk17`	Image officielle LTS avec Java 17.

Image Jenkins personnalisée (avec le client Docker)

```
FROM jenkins/jenkins:lts-jdk17
USER root
RUN apt-get update && apt-get install -y docker.io maven git \
    && rm -rf /var/lib/apt/lists/*
RUN groupadd -f docker && usermod -aG docker jenkins
USER jenkins
```

```
docker build -t jenkins-custom .
docker rm -f jenkins
docker run -d --name jenkins -p 8080:8080 -p 50000:50000 \
  -v jenkins_home:/var/jenkins_home \
  -v /var/run/docker.sock:/var/run/docker.sock \
  --group-add $(stat -c '%g' /var/run/docker.sock) jenkins-custom
```

`USER root` permet d'installer des paquets ; `usermod -aG docker jenkins` ajoute l'utilisateur au groupe `docker` ; `--group-add` évite l'erreur *permission denied* sur le socket.

Installer Jenkins sur Ubuntu

```
sudo apt update
sudo apt install -y fontconfig openjdk-17-jre
java -version

sudo wget -O /usr/share/keyrings/jenkins-keyring.asc \
  https://pkg.jenkins.io/debian-stable/jenkins.io-2023.key
echo "deb [signed-by=/usr/share/keyrings/jenkins-keyring.asc] \
https://pkg.jenkins.io/debian-stable binary/" | \
  sudo tee /etc/apt/sources.list.d/jenkins.list > /dev/null

sudo apt update
sudo apt install -y jenkins
sudo systemctl enable --now jenkins
sudo systemctl status jenkins
```

`openjdk-17-jre` : Java requis ; la clé GPG authentifie les paquets ; `enable --now` active le démarrage automatique *et* lance le service. Pare-feu : `sudo ufw allow 8080/tcp`.

Assistant de premier démarrage

1. Ouvrir `http://localhost:8080`
2. Saisir le mot de passe administrateur initial
3. Choisir **Install suggested plugins**
4. Créer le premier administrateur
5. Confirmer l'URL de Jenkins
6. **Start using Jenkins**

```
# Docker
docker exec jenkins cat \
  /var/jenkins_home/secrets/initialAdminPassword

# Ubuntu
sudo cat \
  /var/lib/jenkins/secrets/initialAdminPassword
```

Plugins nécessaires

Menu : **Manage Jenkins** → **Plugins** → **Available plugins**

Pipeline	Support des pipelines et du Jenkinsfile.
Git / GitHub Integration	Récupération du code, webhooks, déclencheur `githubPush`.
Docker Pipeline	Syntaxe `agent { docker {...} }`.
Credentials Binding	Étape `withCredentials`.
SSH Agent	Étape `sshagent` pour les connexions SSH.
JUnit	Publication des résultats de tests.
Timestamper	Option `timestamps()`.
Workspace Cleanup	Étape `cleanWs()`.

Outils et credentials

Manage Jenkins → Tools

- JDK : nom `JDK-17`
- Maven : nom `Maven-3.9`

Attention

Le nom doit être **identique** à celui du Jenkinsfile (`tools { maven 'Maven-3.9' }`), sinon : *Tool type does not have an install of…*

Manage Jenkins → Credentials

ID	Type
`dockerhub-creds`	Username + password / token
`github-creds`	Username + Personal Access Token
`deploy-ssh-key`	SSH Username + clé privée
`slack-token`	Secret text

Plan

1. Définitions : CI/CD et Jenkins

2. Démarche du lab

3. Installation et configuration

4. Les pipelines

5. Le Jenkinsfile en détail

6. Labs pratiques

7. Fonctions avancées

8. Sécurité et bonnes pratiques

9. Commandes de référence

10. Dépannage

Déclaratif ou scripté ?

	Déclaratif	Scripté
Racine	`pipeline { ... }`	`node { ... }`
Structure	Rigide, sections prédéfinies	Libre, Groovy complet
Lisibilité	Excellente (recommandé)	Plus technique
Validation	Avant l'exécution	À l'exécution
Erreurs	Bloc `post`	`try/catch/finally`
Usage	95 % des cas	Logique très dynamique

Ce lab utilise la syntaxe déclarative.

Créer un job Pipeline dans Jenkins

1. **New Item** → nom `demo-app-pipeline`
2. Type **Pipeline** → OK
3. *Definition* : **Pipeline script from SCM**
4. *SCM* : Git, URL du dépôt, credentials si privé
5. *Branch* : `*/main`
6. *Script Path* : `Jenkinsfile`
7. **Save** puis **Build Now**

Pipeline as Code

Le Jenkinsfile est dans Git : versionné, relu en revue, historisé et évolue avec le code.

Cycle d'exécution d'un build



`SUCCESS`	Tout s'est bien passé.
`UNSTABLE`	Build terminé mais tests en échec ou avertissements.
`FAILURE`	Une étape a échoué.
`ABORTED`	Interrompu manuellement ou par un timeout.
`NOT_BUILT`	Étape non exécutée (condition `when` fausse).

Plan

1. Définitions : CI/CD et Jenkins

2. Démarche du lab

3. Installation et configuration

4. Les pipelines

5. Le Jenkinsfile en détail

6. Labs pratiques

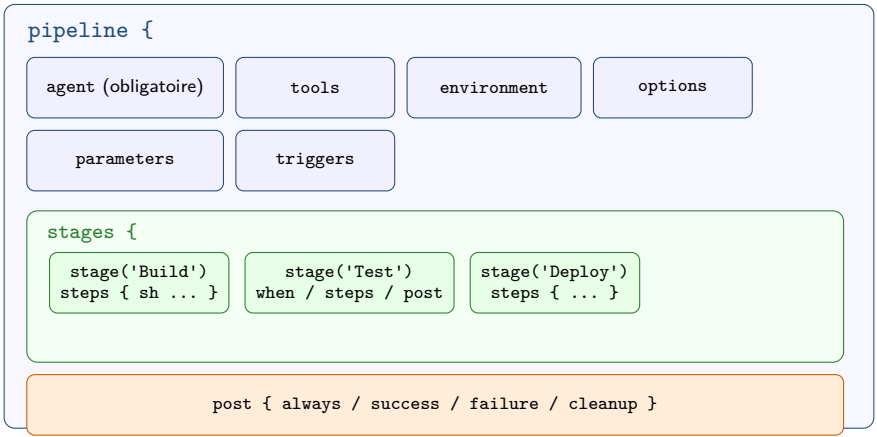
7. Fonctions avancées

8. Sécurité et bonnes pratiques

9. Commandes de référence

10. Dépannage

Structure d'un Jenkinsfile déclaratif



Jenkinsfile de référence (1/3) : en-tête

```
pipeline {
    agent any
    tools { maven 'Maven-3.9'; jdk 'JDK-17' }
    environment {
        APP_NAME = 'demo-app'
        VERSION  = "1.0.${BUILD_NUMBER}"
    }
    options {
        timestamps()
        timeout(time: 15, unit: 'MINUTES')
        buildDiscarder(logRotator(numToKeepStr: '10'))
        disableConcurrentBuilds()
    }
    parameters {
        string(name: 'BRANCH', defaultValue: 'main', description: 'Branche')
        booleanParam(name: 'RUN_TESTS', defaultValue: true, description: 'Tests ?')
    }
    triggers { pollSCM('H/5 * * * *') }
```

Jenkinsfile de référence (2/3) : les stages

```
stages {
    stage('Checkout') {
        steps {
            checkout scm
            sh 'git log -1 --oneline'
        }
    }
    stage('Build') {
        steps { sh 'mvn -B clean compile' }
    }
    stage('Test') {
        when { expression { params.RUN_TESTS } }
        steps { sh 'mvn -B test' }
        post {
            always { junit 'target/surefire-reports/*.xml' }
        }
    }
    stage('Package') {
        steps {
            sh 'mvn -B package -DskipTests'
            archiveArtifacts artifacts: 'target/*.jar', fingerprint: true
        }
    }
}
```

Jenkinsfile de référence (3/3) : post

```
    post {
        success { echo "Build ${VERSION} reussi" }
        failure { echo 'Build en echec' }
        always  { cleanWs() }
    }
}
```

- Le bloc `post` s'exécute après les stages, selon le résultat.
- `always` : toujours ; `success` : si réussi ; `failure` : si échec.
- `cleanWs()` libère le workspace (espace disque).
- Les guillemets doubles permettent l'interpolation `${VERSION}`.

La directive `agent`

```
agent any                           // n'importe quel agent disponible
agent none                          // chaque stage definit son agent
agent { label 'linux && docker' }   // un agent ayant ces labels
agent {
    docker {                        // execute dans un conteneur
        image 'maven:3.9-eclipse-temurin-17'
        args  '-v $HOME/.m2:/root/.m2'
    }
}
agent { dockerfile { filename 'Dockerfile.build' } }
```

- `agent` est obligatoire : il indique où s'exécute le code.
- `docker` : conteneur éphémère, aucun outil à installer sur l'agent.
- `args` : options passées à `docker run` (ici cache Maven monté).

tools et environment

```
tools {
    maven 'Maven-3.9'
    jdk   'JDK-17'
}
environment {
    APP_NAME     = 'demo-app'                    // litteral
    VERSION      = "1.0.${BUILD_NUMBER}"         // interpolation Groovy
    DOCKER_CREDS = credentials('dockerhub-creds')
}
```

- `tools` ajoute Maven et le JDK au `PATH` (noms définis dans Manage Jenkins → Tools).
- **Guillemets simples** : texte littéral ; **doubles** : interpolation.
- `credentials('id')` crée `DOCKER_CREDS`, `DOCKER_CREDS_USR` et `DOCKER_CREDS_PSW` (masqués `****` dans les logs).

La directive `options`

`timestamps()`	Heure devant chaque ligne du log.
`timeout(time: 15, unit: 'MINUTES')`	Interrompt le build après 15 minutes.
`buildDiscarder(logRotator(numToKeepStr: '10'))`	Garde seulement les 10 derniers builds.
`disableConcurrentBuilds()`	Interdit deux builds simultanés du job.
`skipDefaultCheckout()`	Désactive le checkout automatique.
`retry(3)`	Relance tout le pipeline jusqu'à 3 fois.

parameters et triggers

```
parameters {
    string(name: 'BRANCH', defaultValue: 'main', description: 'Branche')
    booleanParam(name: 'RUN_TESTS', defaultValue: true, description: 'Lancer les tests')
    choice(name: 'ENV', choices: ['dev', 'staging', 'prod'], description: 'Cible')
}
triggers {
    cron('H 2 * * *')          // chaque nuit vers 2h
    pollSCM('H/5 * * * *')     // interroge Git toutes les ~5 min
    // githubPush()            // webhook GitHub (plus efficace)
}
```

- Accès : `params.ENV` ; dans `sh` : `$ENV`. Le bouton devient **Build with Parameters**.
- Le `H` (hash) répartit la charge entre jobs.
- Au premier build après ajout de `parameters`, les valeurs par défaut sont utilisées.

Syntaxe cron

Cinq champs : `MINUTE HEURE JOUR_DU_MOIS MOIS JOUR_DE_SEMAINE`

`H/15 * * * *`	Toutes les 15 minutes environ.
`H 2 * * *`	Chaque jour vers 2h00.
`H 8 * * 1-5`	Du lundi au vendredi vers 8h.
`H H(0-5) * * 0`	Dimanche, à une heure aléatoire entre 0h et 5h.
`* * * * *`	Toutes les minutes (à éviter).

Les steps les plus utilisés

`echo 'message'`	Affiche un message dans le log.
`sh 'commande'`	Commande shell (échoue si le code retour $\neq 0$).
`bat 'commande'`	Équivalent de `sh` sous Windows.
`checkout scm`	Récupère le code du dépôt configuré dans le job.
`dir('dossier') { ... }`	Exécute dans un sous-répertoire.
`withCredentials([...]) { }`	Injecte temporairement des secrets.
`sshagent(['id']) { }`	Charge une clé SSH le temps du bloc.
`junit 'chemin/*.xml'`	Publie les résultats de tests.
`archiveArtifacts artifacts: '...'`	Conserve des fichiers produits.
`stash` / `unstash`	Transmet des fichiers entre stages ou agents.
`timeout` / `retry(3)` / `sleep`	Limite, relance, pause.
`input message: '...'`	Attend une validation manuelle.
`script { ... }`	Bloc Groovy libre dans un pipeline déclaratif.
`cleanWs()`	Vide le workspace.

Le step `sh` en détail

```groovy
sh 'mvn -B clean package'                          // une commande

sh '''
    echo "Debut"
    mvn -B test
    echo "Fin"
'''                                                 // plusieurs commandes

def sortie = sh(script: 'git rev-parse --short HEAD', returnStdout: true).trim()
def code   = sh(script: 'ls /inexistant', returnStatus: true)   // n'echoue pas
```

- Par défaut : `/bin/sh -xe` : chaque commande est affichée (`-x`) et le script s'arrête à la première erreur (`-e`).
- `returnStdout: true` renvoie la sortie ; `returnStatus: true` renvoie le code de retour.
- Les triples guillemets simples `'''` évitent l'interpolation (recommandé avec des secrets).

La condition `when`

```
stage('Deploy to prod') {
    when { branch 'main' }                        // seulement sur main
    steps { echo 'Deploiement en production' }
}
stage('Condition combinee') {
    when {
        allOf {
            branch 'main'
            expression { params.ENV == 'prod' }
            not { changeRequest() }
        }
    }
    steps { echo 'OK' }
}
```

Conditions : `branch`, `environment`, `expression`, `tag`, `changeRequest`, `allOf`, `anyOf`, `not`. Condition fausse : le stage est *sauté* (gris).

Le bloc `post`

`always`	Toujours, quel que soit le résultat.
`success`	Uniquement si le build réussit.
`failure`	Uniquement si le build échoue.
`unstable`	Si le build est instable (tests en échec).
`changed`	Si le résultat diffère du build précédent.
`fixed`	Réussi alors que le précédent avait échoué.
`regression`	Pire que le build précédent.
`aborted`	Build interrompu manuellement.
`cleanup`	Toujours, *en dernier*, après tous les autres blocs.

Peut se placer au niveau du pipeline ou d'un `stage`.

Variables d'environnement prédéfinies

`BUILD_NUMBER`	Numéro du build courant.
`BUILD_URL`	URL du build dans Jenkins.
`JOB_NAME`	Nom du job.
`WORKSPACE`	Chemin absolu du workspace.
`JENKINS_URL`	URL de l'instance Jenkins.
`NODE_NAME`	Agent qui exécute le build.
`BRANCH_NAME`	Nom de la branche (jobs multibranch).
`GIT_COMMIT`	Hash complet du commit.
`GIT_BRANCH`	Branche Git (ex. `origin/main`).
`CHANGE_ID`	Numéro de la pull request (multibranch).

Plan

1. Définitions : CI/CD et Jenkins

2. Démarche du lab

3. Installation et configuration

4. Les pipelines

5. Le Jenkinsfile en détail

6. Labs pratiques

7. Fonctions avancées

8. Sécurité et bonnes pratiques

9. Commandes de référence

10. Dépannage

Lab 1 : « Hello Jenkins »

```
pipeline {
    agent any
    stages {
        stage('Hello') {
            steps {
                echo 'Hello Jenkins !'
                sh 'whoami'
                sh 'pwd'
                sh 'date'
            }
        }
        stage('Environnement') {
            steps {
                echo "Job : ${env.JOB_NAME} #${env.BUILD_NUMBER}"
                sh 'printenv | sort | head -20'
            }
        }
    }
}
```

Étapes

1. New Item → `lab1-hello`, type Pipeline
2. Coller le script (*Pipeline script*)
3. Save puis **Build Now**
4. Build #1 → **Console Output**

Explications

- `whoami` : utilisateur (`jenkins`)
- `pwd` : workspace
- `printenv` : variables du build

Lab 2 : le code de l'application

```java
package com.example;
import com.sun.net.httpserver.HttpServer;
import java.net.InetSocketAddress;
import java.nio.charset.StandardCharsets;

public class App {
    public static String message() { return "Hello from Jenkins CI/CD!"; }

    public static void main(String[] args) throws Exception {
        HttpServer server = HttpServer.create(new InetSocketAddress(8080), 0);
        server.createContext("/", exchange -> {
            byte[] body = message().getBytes(StandardCharsets.UTF_8);
            exchange.sendResponseHeaders(200, body.length);
            exchange.getResponseBody().write(body);
            exchange.close();
        });
        server.start();
    }
}
```

```java
public class AppTest {   // JUnit 4
    @Test public void messageIsCorrect() {
        assertEquals("Hello from Jenkins CI/CD!", App.message());
    }
}
```

Lab 2 : pom.xml et premier push

```
<properties>
  <maven.compiler.source>17</maven.compiler.source>
  <maven.compiler.target>17</maven.compiler.target>
  <project.build.sourceEncoding>UTF-8</project.build.sourceEncoding>
</properties>
<build><plugins><plugin>
  <groupId>org.apache.maven.plugins</groupId>
  <artifactId>maven-jar-plugin</artifactId>
  <configuration><archive><manifest>
    <mainClass>com.example.App</mainClass>
  </manifest></archive></configuration>
</plugin></plugins></build>
```

```
mvn -B clean package          # doit reussir en local avant de pousser
git add . && git commit -m "Application HTTP + test"
git remote add origin https://github.com/VOTRE_COMPTE/demo-app.git
git branch -M main
git push -u origin main
```

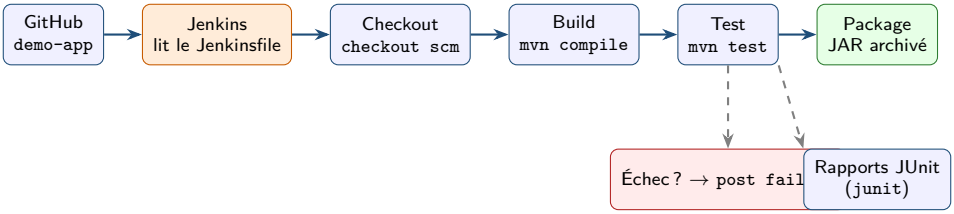
Le manifeste `mainClass` rend le JAR exécutable ; Java 17 évite l'erreur de compilation de l'archétype par défaut.

Lab 2 : commandes Maven utilisées

`mvn -B clean compile`	`-B` (batch) : logs lisibles dans Jenkins. `clean` supprime `target/`, `compile` compile.
`mvn -B test`	Tests unitaires (Surefire). Rapports dans `target/surefire-reports/`.
`mvn -B package -DskipTests`	Construit le JAR sans relancer les tests.
`mvn -B verify`	Va plus loin que `package` : tests d'intégration.
`mvn -B deploy`	Publie l'artefact dans Nexus / Artifactory.

Résultats à observer : Stage View avec 4 colonnes vertes, graphique JUnit (Test Result), JAR téléchargeable dans *Build Artifacts*.

Lab 2 : schéma du pipeline CI



Expérience : cassez volontairement un test → stage Test rouge, Package ignoré, `post { failure }` exécuté. Corrigez ensuite.

Lab 3 : le Dockerfile

```
FROM eclipse-temurin:17-jre-alpine
WORKDIR /app
COPY target/*.jar app.jar
EXPOSE 8080
ENTRYPOINT ["java", "-jar", "app.jar"]
```

- `FROM` : image de base légère avec JRE 17.
- `WORKDIR /app` : répertoire de travail dans l'image.
- `COPY target/*.jar app.jar` : copie le JAR produit par Maven.
- `EXPOSE 8080` : documente le port (n'ouvre rien à lui seul).
- `ENTRYPOINT` : commande lancée au démarrage du conteneur.

Lab 3 : build et push Docker (1/2)

```
pipeline {
    agent any
    tools { maven 'Maven-3.9'; jdk 'JDK-17' }
    environment {
        IMAGE        = 'votrecompte/demo-app'
        TAG          = "${BUILD_NUMBER}"
        DOCKER_CREDS = credentials('dockerhub-creds')
    }
    options { timestamps(); timeout(time: 20, unit: 'MINUTES') }

    stages {
        stage('Checkout') { steps { checkout scm } }
        stage('Build & Test') {
            steps { sh 'mvn -B clean package' }
            post { always { junit 'target/surefire-reports/*.xml' } }
        }
        stage('Docker Build') {
            steps { sh 'docker build -t $IMAGE:$TAG -t $IMAGE:latest .' }
        }
```

Lab 3 : build et push Docker (2/2)

```
        stage('Docker Push') {
            when { branch 'main' }
            steps {
                sh 'echo "$DOCKER_CREDS_PSW" | docker login -u "$DOCKER_CREDS_USR" --password-stdin'
                sh 'docker push $IMAGE:$TAG'
                sh 'docker push $IMAGE:latest'
            }
            post { always { sh 'docker logout' } }
        }
    }
    post { always { cleanWs() } }
}
```

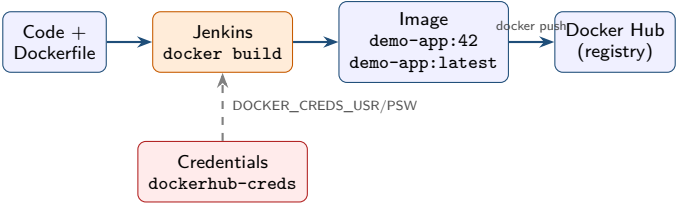
- `docker build -t ... -t ... .` : deux tags (n° de build + `latest`) ; le `.` final = contexte courant.
- **Guillemets simples** : le shell remplace les variables, le secret n'est pas interpolé par Groovy.
- `--password-stdin` : mot de passe via l'entrée standard (invisible dans `ps`).
- `when { branch 'main' }` : nécessite un job Multibranch ; sinon tester `env.GIT_BRANCH`.

Lab 3 : vérification de l'image

```
docker images | grep demo-app
docker run -d --name test-app -p 9090:8080 votrecompte/demo-app:latest
curl http://localhost:9090           # -> Hello from Jenkins CI/CD!
docker logs test-app
docker rm -f test-app
```

`docker images`	Liste les images locales.
`docker run -d -p 9090:8080`	Conteneur en arrière-plan, port 9090 hôte → 8080 conteneur.
`curl http://localhost:9090`	Teste l'application.
`docker logs NOM`	Sortie du conteneur.
`docker rm -f NOM`	Arrête et supprime le conteneur.

Lab 3 : schéma build et push



Lab 4 : préparer le serveur cible (clé SSH)

```
ssh-keygen -t ed25519 -f jenkins_deploy -C "jenkins-deploy" -N ""
ssh-copy-id -i jenkins_deploy.pub deploy@192.168.1.50
ssh -i jenkins_deploy deploy@192.168.1.50 "docker --version"
```

- `ssh-keygen -t ed25519` : paire de clés moderne ; `-f` nom du fichier ; `-N ""` sans passphrase (automatisation).
- `ssh-copy-id` : installe la clé publique dans `authorized_keys` du serveur.
- Ajouter la clé *privée* `jenkins_deploy` dans Jenkins : credential **SSH Username with private key**, ID `deploy-ssh-key`.

Lab 4 : stages de déploiement

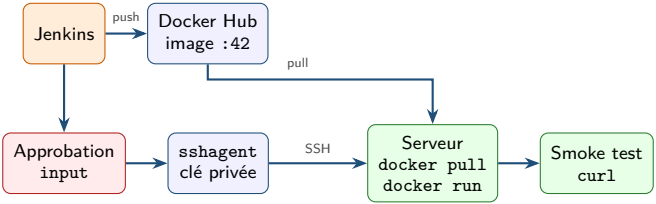
```
stage('Approbation') {
    when { branch 'main' }
    steps {
        timeout(time: 10, unit: 'MINUTES') {
            input message: 'Deployer en production ?', ok: 'Deployer'
        }
    }
}
stage('Deploy') {
    when { branch 'main' }
    steps {
        sshagent(credentials: ['deploy-ssh-key']) {
            sh '''
                ssh -o StrictHostKeyChecking=no deploy@192.168.1.50 "
                    docker pull $IMAGE:$TAG &&
                    docker rm -f demo-app || true &&
                    docker run -d --name demo-app --restart unless-stopped \
                        -p 80:8080 $IMAGE:$TAG
                "
            '''
        }
    }
}
```

Lab 4 : smoke test et explications

```
stage('Smoke test') {
    when { branch 'main' }
    steps {
        retry(5) {
            sleep time: 3, unit: 'SECONDS'
            sh 'curl -fsS http://192.168.1.50/ | grep "Hello"'
        }
    }
}
```

- `input` met le pipeline en pause (entouré d'un `timeout`).
- `sshagent` charge la clé le temps du bloc, puis la supprime.
- `docker rm -f ... || true` : pas d'échec si le conteneur n'existe pas.
- `curl -fsS` : échoue si HTTP ≥ 400 ; `retry(5)` laisse l'appli démarrer.
- `StrictHostKeyChecking=no` : lab uniquement, préférer `known_hosts`.

Lab 4 : schéma du déploiement



Lab 5 : pipeline final (1/3) : configuration

```groovy
pipeline {
    agent any
    tools { maven 'Maven-3.9'; jdk 'JDK-17' }
    parameters {
        choice(name: 'ENV', choices: ['dev', 'staging', 'prod'], description: 'Cible')
        booleanParam(name: 'SKIP_TESTS', defaultValue: false, description: 'Sauter les tests')
    }
    environment {
        IMAGE        = 'votrecompte/demo-app'
        VERSION      = "1.0.${BUILD_NUMBER}"
        DOCKER_CREDS = credentials('dockerhub-creds')
    }
    options {
        timestamps()
        timeout(time: 30, unit: 'MINUTES')
        buildDiscarder(logRotator(numToKeepStr: '15', artifactNumToKeepStr: '5'))
        disableConcurrentBuilds()
    }
    triggers { githubPush() }
```

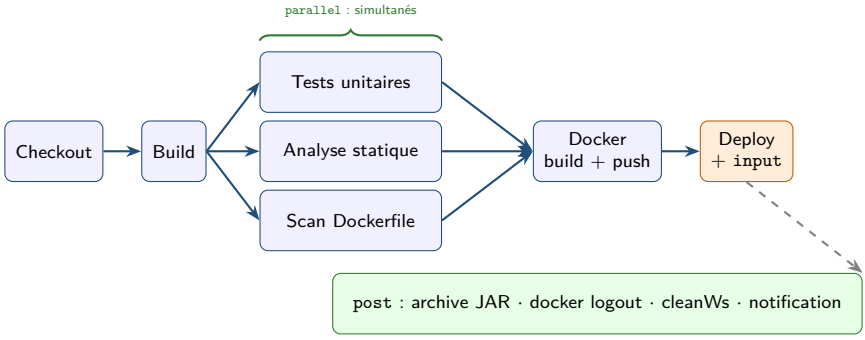
Lab 5 : pipeline final (2/3) : parallélisme

```
stages {
    stage('Checkout') {
        steps {
            checkout scm
            script { env.GIT_SHORT = sh(script: 'git rev-parse --short HEAD',
                                        returnStdout: true).trim() }
            echo "Commit : ${env.GIT_SHORT}"
        }
    }
    stage('Build') { steps { sh 'mvn -B clean package -DskipTests' } }
    stage('Qualite') {
        when { expression { !params.SKIP_TESTS } }
        parallel {
            stage('Tests unitaires') {
                steps { sh 'mvn -B test' }
                post { always { junit 'target/surefire-reports/*.xml' } }
            }
            stage('Analyse statique') { steps { sh 'mvn -B verify -DskipTests' } }
            stage('Scan Dockerfile') { steps { sh 'grep -n "FROM" Dockerfile' } }
        }
    }
```

Lab 5 : pipeline final (3/3) : docker, deploy, post

```
        stage('Docker') {
            steps {
                sh 'docker build -t $IMAGE:$VERSION .'
                sh 'echo "$DOCKER_CREDS_PSW" | docker login -u "$DOCKER_CREDS_USR" --password-stdin'
                sh 'docker push $IMAGE:$VERSION'
            }
        }
        stage('Deploy') {
            when { expression { params.ENV != 'dev' } }
            steps {
                input message: "Deployer ${VERSION} sur ${params.ENV} ?"
                echo "Deploiement de ${VERSION} vers ${params.ENV}"
            }
        }
    }
    post {
        success { archiveArtifacts artifacts: 'target/*.jar', fingerprint: true }
        failure { echo "ECHEC - ${env.BUILD_URL}console" }
        always  { sh 'docker logout || true'; cleanWs() }
    }
}
```

Lab 5 : schéma du pipeline final



Plan

1. Définitions : CI/CD et Jenkins

2. Démarche du lab

3. Installation et configuration

4. Les pipelines

5. Le Jenkinsfile en détail

6. Labs pratiques

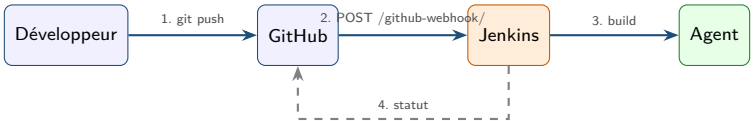
7. Fonctions avancées

8. Sécurité et bonnes pratiques

9. Commandes de référence

10. Dépannage

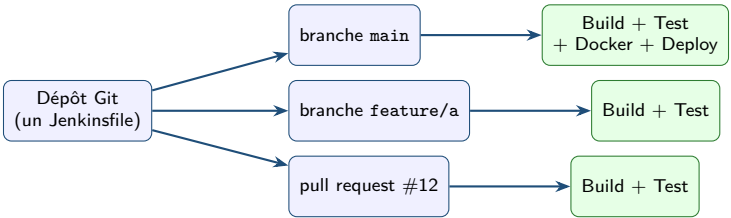
Webhook GitHub : déclencher à chaque push



1. GitHub : **Settings** → **Webhooks** → **Add webhook**
2. Payload URL : `http://VOTRE_JENKINS:8080/github-webhook/` (le `/` final compte)
3. Content type : `application/json` ; événement *push*
4. Jenkinsfile : `triggers { githubPush() }`

Poste local : exposer temporairement avec `ngrok http 8080`.

Pipeline Multibranch



New Item → **Multibranch Pipeline** → Branch Sources (Git/GitHub) → *by Jenkinsfile*. Ici `branch 'main'`, `BRANCH_NAME` et `changeRequest()` fonctionnent : déploiement uniquement depuis `main`.

Agents Docker : environnement propre à chaque build

```
pipeline {
    agent none
    stages {
        stage('Build Java') {
            agent { docker { image 'maven:3.9-eclipse-temurin-17'
                             args  '-v $HOME/.m2:/root/.m2' } }
            steps { sh 'mvn -B clean package' }
        }
        stage('Tests front') {
            agent { docker { image 'node:20-alpine' } }
            steps { sh 'node --version'; sh 'npm ci && npm test' }
        }
    }
}
```

`agent none` : chaque stage choisit son environnement. Jenkins démarre le conteneur, y monte le workspace, exécute les steps puis le supprime : aucun outil à installer, versions reproductibles.

Matrix et gestion des erreurs

```
matrix {
    axes {
        axis {
            name 'JAVA_VERSION'
            values '11', '17', '21'
        }
        axis {
            name 'OS'
            values 'linux', 'windows'
        }
    }
    stages {
        stage('Test') {
            steps { echo "Java ${JAVA_VERSION}/${OS}" }
        }
    }
}
```

$3 \times 2 = 6$ combinaisons en parallèle.

```
catchError(buildResult: 'UNSTABLE',
           stageResult: 'FAILURE') {
    sh 'mvn -B verify'
}
retry(3) {
    sh 'curl -fsS https://exemple.com/health'
}
script {
    try { sh 'commande-risquee' }
    catch (err) {
        echo "Erreur : ${err}"
        currentBuild.result = 'UNSTABLE'
    }
}
```

`catchError` : stage rouge mais le pipeline continue.

Notifications, stash et agents distants

```
post {
    failure {
        mail to: 'equipe@exemple.com',
             subject: "ECHEC ${env.JOB_NAME}",
             body: "${env.BUILD_URL}"
    }
}
// stash / unstash
stash name: 'app', includes: 'target/*.jar'
unstash 'app'
```

Nœud distant

1. Manage Jenkins → Nodes → New Node
2. Labels (ex. `linux docker`), exécuteurs
3. Sur l'agent :

```
java -jar agent.jar -url http://JENKINS:8080/ \
  -secret SECRET -name agent-linux-1 \
  -webSocket -workDir /home/jenkins/agent
```

Ciblage : `agent { label 'linux' }`

Shared Libraries et pipeline scripté

Bibliothèque partagée

```groovy
// vars/buildApp.groovy
def call(String imageName) {
    sh 'mvn -B clean package'
    sh "docker build -t ${imageName}:${env.BUILD_NUMBER}
       ."
}

// Jenkinsfile
@Library('mes-libs@main') _
pipeline {
    agent any
    stages { stage('Build') {
        steps { buildApp('compte/demo-app') } } }
}
```

Pipeline scripté

```groovy
node {
    try {
        stage('Checkout') { checkout scm }
        stage('Build') { sh 'mvn -B clean package' }
        stage('Test') {
            sh 'mvn -B test'
            junit 'target/surefire-reports/*.xml'
        }
    } catch (e) {
        currentBuild.result = 'FAILURE'
        throw e
    } finally { cleanWs() }
}
```

Plan

1. Définitions : CI/CD et Jenkins

2. Démarche du lab

3. Installation et configuration

4. Les pipelines

5. Le Jenkinsfile en détail

6. Labs pratiques

7. Fonctions avancées

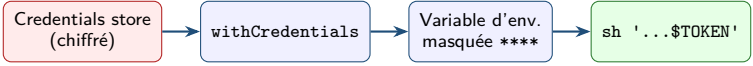
8. Sécurité et bonnes pratiques

9. Commandes de référence

10. Dépannage

Gérer les secrets correctement

```
// MAUVAIS : interpolation Groovy d'un secret (visible dans les logs)
withCredentials([string(credentialsId: 'api-token', variable: 'TOKEN')]) {
    sh "curl -H 'Authorization: Bearer ${TOKEN}' https://api.exemple.com"
}
// BON : guillemets simples, le shell lit la variable d'environnement
withCredentials([string(credentialsId: 'api-token', variable: 'TOKEN')]) {
    sh 'curl -H "Authorization: Bearer $TOKEN" https://api.exemple.com'
}
// Username / password
withCredentials([usernamePassword(credentialsId: 'github-creds',
        usernameVariable: 'GH_USER', passwordVariable: 'GH_PASS')]) {
    sh 'git clone https://$GH_USER:$GH_PASS@github.com/org/repo.git'
}
```



Bonnes pratiques

- **Pipeline as Code** : un Jenkinsfile par dépôt, relu en PR
- **Contrôleur léger** : 0 exécuteur, builds sur des agents
- **Accès** : authentification, Role-Based, moindre privilège
- **HTTPS** via reverse proxy (Nginx, Traefik)
- **Mises à jour** : version LTS, plugins à jour
- **Sauvegarde** de `JENKINS_HOME`
- **Builds propres** : `cleanWs()` + `buildDiscarder`
- **Toujours un** `timeout`, même autour de `input`
- **Versions figées** : tags d'images explicites
- **Approbation manuelle** avant la production

Sauvegarde et restauration

```
# Docker : sauvegarde du volume
docker run --rm -v jenkins_home:/data -v $(pwd):/backup alpine \
    tar czf /backup/jenkins_home_$(date +%F).tar.gz -C /data .

# Docker : restauration
docker run --rm -v jenkins_home:/data -v $(pwd):/backup alpine \
    tar xzf /backup/jenkins_home_2025-01-15.tar.gz -C /data

# Installation Ubuntu
sudo systemctl stop jenkins
sudo tar czf /tmp/jenkins_backup.tar.gz -C /var/lib/jenkins .
sudo systemctl start jenkins
```

`tar czf` crée une archive compressée ; `-C` change de répertoire avant d'archiver ; `--rm` supprime le conteneur temporaire.

Plan

1. Définitions : CI/CD et Jenkins

2. Démarche du lab

3. Installation et configuration

4. Les pipelines

5. Le Jenkinsfile en détail

6. Labs pratiques

7. Fonctions avancées

8. Sécurité et bonnes pratiques

9. Commandes de référence

10. Dépannage

Service Jenkins (Ubuntu)

`sudo systemctl start jenkins`	Démarre le service.
`sudo systemctl stop jenkins`	Arrête le service.
`sudo systemctl restart jenkins`	Redémarre le service.
`sudo systemctl status jenkins`	Affiche l'état.
`sudo systemctl enable jenkins`	Démarrage automatique au boot.
`sudo journalctl -u jenkins -f`	Suit les logs en direct.
`sudo ss -tlnp`	Ports en écoute (chercher 8080).
`sudo -u jenkins -s`	Shell avec l'utilisateur jenkins.

Docker : commandes essentielles

`docker ps` (`-a` : tous)	Conteneurs en cours.
`docker logs -f jenkins`	Suit les logs du conteneur.
`docker exec -it jenkins bash`	Ouvre un shell dans le conteneur.
`docker restart jenkins`	Redémarre le conteneur.
`docker volume ls`	Liste les volumes.
`docker build -t nom:tag .`	Construit une image.
`docker tag nom:tag compte/nom:tag`	Ajoute un nom à l'image.
`docker login` / `docker push`	Authentification / envoi vers le registry.
`docker system prune -af`	Nettoie images et caches inutilisés.
`docker compose up -d`	Démarre une pile `docker-compose.yml`.

Git et Maven

Git	
`git clone URL`	Copie un dépôt.
`git status`	État local.
`git add .`	Ajoute à l'index.
`git commit -m "msg"`	Enregistre.
`git push origin main`	Envoie sur `main`.
`git pull`	Récupère et fusionne.
`git checkout -b feat/x`	Nouvelle branche.
`git log --oneline -5`	5 derniers commits.
`git rev-parse --short HEAD`	Hash court.

Maven	
`mvn -v`	Versions Maven/JDK.
`mvn clean`	Supprime `target/`.
`mvn compile`	Compile.
`mvn test`	Tests unitaires.
`mvn package`	Construit le JAR.
`mvn verify`	+ tests d'intégration.
`mvn -B`	Mode batch (CI).
`mvn -DskipTests`	Saute les tests.
`mvn dependency:tree`	Arbre des dépendances.

Jenkins CLI

```
wget http://localhost:8080/jnlpJars/jenkins-cli.jar

export JENKINS_URL=http://localhost:8080/
export JENKINS_USER_ID=admin
export JENKINS_API_TOKEN=xxxxxxxxxxxxxxxx      # Utilisateur -> Security -> Add new Token

java -jar jenkins-cli.jar -s $JENKINS_URL -auth $JENKINS_USER_ID:$JENKINS_API_TOKEN list-jobs
java -jar jenkins-cli.jar -s $JENKINS_URL -auth $JENKINS_USER_ID:$JENKINS_API_TOKEN build lab1-hello -s -v
java -jar jenkins-cli.jar -s $JENKINS_URL -auth $JENKINS_USER_ID:$JENKINS_API_TOKEN install-plugin git
java -jar jenkins-cli.jar -s $JENKINS_URL -auth $JENKINS_USER_ID:$JENKINS_API_TOKEN safe-restart
```

`list-jobs`	Liste les jobs.
`build JOB -s -v`	Lance un build ; `-s` attend la fin, `-v` affiche le log.
`install-plugin NOM`	Installe un plugin.
`safe-restart`	Redémarre après la fin des builds en cours.

API REST et validation d'un Jenkinsfile

```
# Lancer un build
curl -X POST -u admin:TOKEN http://localhost:8080/job/lab1-hello/build

# Build parametre
curl -X POST -u admin:TOKEN \
  "http://localhost:8080/job/lab5/buildWithParameters?ENV=staging&SKIP_TESTS=false"

# Etat du dernier build (JSON)
curl -s -u admin:TOKEN http://localhost:8080/job/lab1-hello/lastBuild/api/json | head

# Valider la syntaxe d'un Jenkinsfile AVANT de pousser
curl -X POST -u admin:TOKEN -F "jenkinsfile=<Jenkinsfile" \
  http://localhost:8080/pipeline-model-converter/validate
```

La dernière commande renvoie *Jenkinsfile successfully validated.* ou la liste des erreurs, sans exécuter le pipeline.

Commandes Linux de diagnostic

`whoami` / `id`	Utilisateur courant et ses groupes.
`pwd` / `ls -la`	Dossier courant / contenu détaillé.
`df -h` / `free -h`	Espace disque / mémoire.
`printenv | sort`	Variables d'environnement.
`which mvn java docker`	Chemin des exécutables.
`chmod +x script.sh`	Rend un script exécutable.
`curl -fsS URL`	Requête HTTP, échoue sur erreur.
`grep -n "motif" fichier`	Cherche un motif (numéro de ligne).
`tail -n 100 fichier`	100 dernières lignes.

Plan

1. Définitions : CI/CD et Jenkins

2. Démarche du lab

3. Installation et configuration

4. Les pipelines

5. Le Jenkinsfile en détail

6. Labs pratiques

7. Fonctions avancées

8. Sécurité et bonnes pratiques

9. Commandes de référence

10. Dépannage

Erreurs courantes (1/2)

Symptôme	Cause	Solution
`docker: command not found`	Client Docker absent de l'agent.	Image personnalisée ou agent Docker.
`permission denied ... docker.sock`	`jenkins` hors du groupe `docker`.	`usermod -aG docker jenkins` ; `--group-add`.
`mvn: command not found`	Outil absent du `PATH`.	Déclarer `tools { maven '...' }`.
`Tool type "maven" does not have an install of ...`	Nom différent de celui des Tools.	Aligner exactement les noms.
`BRANCH_NAME` nul	Job non multibranch.	Utiliser un Multibranch ou `GIT_BRANCH`.
`unexpected token`	Erreur de syntaxe Groovy.	Valider avec l'endpoint `validate`.

Erreurs courantes (2/2)

Symptôme	Cause	Solution
`Scripts not permitted to use method`	Sandbox Groovy.	Approuver dans In-process Script Approval.
`Host key verification failed`	Empreinte SSH inconnue.	Ajouter à `known_hosts`.
`Authentication failed` (Git)	Token expiré ou erroné.	Recréer le Personal Access Token.
Webhook sans effet	Jenkins inaccessible, mauvaise URL.	Voir *Recent Deliveries* ; URL en `/github-webhook/`.
« Waiting for next available executor »	Aucun exécuteur / label inconnu.	Vérifier agents, labels, exécuteurs.
`No space left on device`	Disque saturé.	`cleanWs()`, `buildDiscarder`, `docker system prune -af`.

Méthode de diagnostic

1. Ouvrir **Console Output** et remonter à la première erreur.
2. Utiliser **Replay** pour modifier et rejouer le Jenkinsfile sans commit.
3. Reproduire la commande fautive avec le même utilisateur que Jenkins :

```
docker exec -it -u jenkins jenkins bash       # conteneur
sudo -u jenkins -s                            # installation Ubuntu
```

4. Ajouter temporairement `sh 'printenv'`, `sh 'ls -la'`, `sh 'which mvn'`.
5. Consulter **Manage Jenkins** → **System Log** ou `journalctl -u jenkins`.

Récapitulatif : les 10 réflexes

1. Jenkinsfile dans Git, à la racine
2. Toujours `timeout` et `buildDiscarder`
3. Guillemets simples pour les secrets
4. Secrets via *Credentials* uniquement
5. Un stage = une responsabilité
6. Publier les tests même en cas d'échec (`post always`)
7. Nettoyer : `cleanWs()`, `docker logout`
8. Valider la syntaxe avant de pousser
9. Agents Docker pour des builds reproductibles
10. Approbation manuelle avant la production



Merci pour votre attention

Des questions ?

Samir Ouhmmou & Taoufik Mansouri